

Cloud-Based Database (RDS) – Documentation

1. Project Title

AWS RDS Database Deployment Using Custom VPC and EC2

2. Objective

The objective of this project is to deploy and manage a MySQL database using **Amazon RDS** within a custom AWS networking environment. The project demonstrates the creation of networking resources, launching an EC2 instance, connecting to an RDS database, and performing database operations using the MariaDB client.

3. AWS Services Used

- **Amazon VPC**
- **Amazon EC2**
- **Amazon RDS (MySQL)**

4. Project Details

- **VPC Name:** smrthivpc
- **Public Subnet:** smpub
- **Private Subnet:** smpriv
- **Route Tables:**
 - smroute1
 - smroute2
- **Internet Gateway:** SMlg
- **EC2 Instance:** smrdsec2
- **AMI:** Amazon Linux
- **Key Pair:** PPK Key
- **Database Identifier:** database-1
- **Database Engine:** MySQL

5. Project Architecture

- **Custom VPC:** smrthivpc
- **Public Subnet:** smpub
- **Private Subnet:** smpriv
- **Route Tables:** smroute1 and smroute2
- **Internet Gateway:** SMlg
- **EC2 Instance:** smrdsec2

- **Amazon RDS Database:** `database-1`

6. Steps Followed to Deploy the Database

Step 1: Create a Custom VPC

1. Open **AWS Management Console**.
2. Navigate to the **VPC Service**.
3. Create a VPC named `smrthivpc`.
4. Configure the required CIDR block.
5. Save the VPC configuration.

Step 2: Create Subnets

1. Create a public subnet named `smpub`.
2. Create a private subnet named `smpriv`.
3. Associate both subnets with `smrthivpc`.

Step 3: Create an Internet Gateway

1. Create an Internet Gateway named `SMIg`.
2. Attach the Internet Gateway to `smrthivpc`.

Step 4: Create Route Tables

1. Create route table `smroute1`.
2. Create route table `smroute2`.
3. Add a route to the Internet Gateway for the public subnet.
4. Associate route tables with their respective subnets.

Step 5: Launch EC2 Instance

1. Navigate to the **EC2 Dashboard**.
2. Launch a new instance named `smrdsec2`.
3. Select **Amazon Linux AMI**.
4. Choose the VPC `smrthivpc`.
5. Select subnet `smpub`.
6. Create and download a **PPK Key Pair**.
7. Configure security group settings.
8. Launch the instance.

Step 6: Create RDS Database

1. Navigate to **Amazon RDS**.
2. Click **Create Database**.
3. Select **MySQL** as the database engine.

4. Choose the required template.
5. Enter database identifier as **database-1**.
6. Configure username and password.
7. Select VPC **smrithivpc**.
8. Configure networking and security settings.
9. Create the database instance.

Step 7: Connect to the EC2 Instance

1. Open **PuTTY**.
2. Load the **PPK Key** file.
3. Enter the public IP address of the EC2 instance.
4. Connect to the instance.

Step 8: Install MariaDB Client

1. Update the system packages.
2. Install the **MariaDB Client** package.
3. Verify the installation.

Step 9: Connect to the RDS Database

1. Use the **RDS Endpoint** to connect to the MySQL database.
2. Provide the database username and password when prompted.

Step 10: Create a Database

1. Create a new database.
2. Select the database for use.

Step 11: Create a Table

1. Create a table to store sample records.

Step 12: Perform CRUD Operations

- **Create:** Insert records into the table.
- **Read:** Retrieve records using SELECT statements.
- **Update:** Modify existing records.
- **Delete:** Remove records from the table.

7. Output

- Successfully created a custom VPC and networking components.
- Launched an **Amazon Linux EC2 Instance**.
- Created an **Amazon RDS MySQL Database Instance**.

- Connected to the database using the **MariaDB Client**.
- Created a database and table.
- Performed **Create, Read, Update, and Delete (CRUD)** operations successfully.

8. Key Features

- Managed relational database service.
- Secure database deployment within a custom VPC.
- Scalable and highly available infrastructure.
- Simplified database administration.
- Integration with EC2 instances for application connectivity.

9. Conclusion

This project demonstrates the deployment of a **MySQL Database** using **Amazon RDS** within a custom AWS networking environment. A custom VPC (**smrithivpc**) was configured with public and private subnets, route tables, and an internet gateway. An **Amazon Linux EC2 Instance (smrdsec2)** was used to connect to the **RDS Database (database-1)** using the **MariaDB Client**, and CRUD operations were successfully performed to verify database connectivity and functionality.